SK네트웍스 Family AI 과정 10기

모델링 및 평가 LLM 활용 소프트웨어

산출물 단계	모델링 및 평가
평가 산출물	LLM 활용 소프트웨어
제출 일자	2025.06.27
깃허브 경로	https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN10-FINAL-3Team
작성 팀원	배민경, 김현수

1. 시스템 개요

본 시스템은 LLM과 sLLM 두 구성 요소로 나뉘며, 각각의 역할에 따라 기능을 분담하여 작동합니다.

LLM은 사용자의 자연어 질문을 받아, Vector DB와 RDB를 함께 연동하는 RAG 구조로 동작합니다. 질문 내용을 분석하여 관련 판례를 검색한 뒤, 채팅 형식의 답변을 생성하여 제공합니다. 이를 통해 단순 키워드 검색이 아닌, 의미 기반의 질의응답이 가능합니다.

sLLM은 FastAPI에 연동된 데이터를 받아, 내부적으로 RDB와 연결되어 필요한 결과를 생성합니다. 생성된 결과는 Django 프레임워크를 통해 웹 페이지에 전달되며, 사용자는 이를 UI 상에서 확인할 수 있습니다. 동시에 Django는 해당 결과를 RDB에 다시 저장하여 데이터의 일관성과 추적성을 유지합니다.

이 전체 과정에서 OpenAI API Key, DB 접속 URL, 비밀번호 등의 민감한 정보는 직접 코드에 넣지 않고 안전하게 관리됩니다. 또한, LLM 응답 품질과 속도를 높이기 위해 프롬프트 구조를 질의 유형별로 최적화하여 시스템 전체의 응답성과 정확도를 향상시켰습니다.

2. 시스템 구성 요소

```
[1] 사용자
        └─ 질문 입력 (웹 프론트엔드 - Django)
        ▼
[2] 백엔드 서버 (FastAPI)
        ├─ 입력 쿼리 수신
        ├─ 쿼리 유형 분류 (조건검색 / 유사도검색 / 일반질문) ← LLM 처리
        ▼
        ┌─ 조건검색인 경우:
        │   └─ RDB 필터 추출
        │   └─ Django 서버로 전달하여 메타데이터 기반 판례 검색
        │   └─ 관련 판례 리스트 수신
        │
        └─ 유사도검색인 경우:
        └─ 질문 임베딩 생성
        └─ FAISS(Vector DB)에서 유사 판례 검색
```

└ 관련 판례 리스트 수신

▼

[3] LLM 처리

├ 검색된 판례 내용을 기반으로 프롬프트 구성

├ 질문 유형(단건/비교/추천)에 따라 응답 생성

└ LLM(OpenAI) 호출

▼

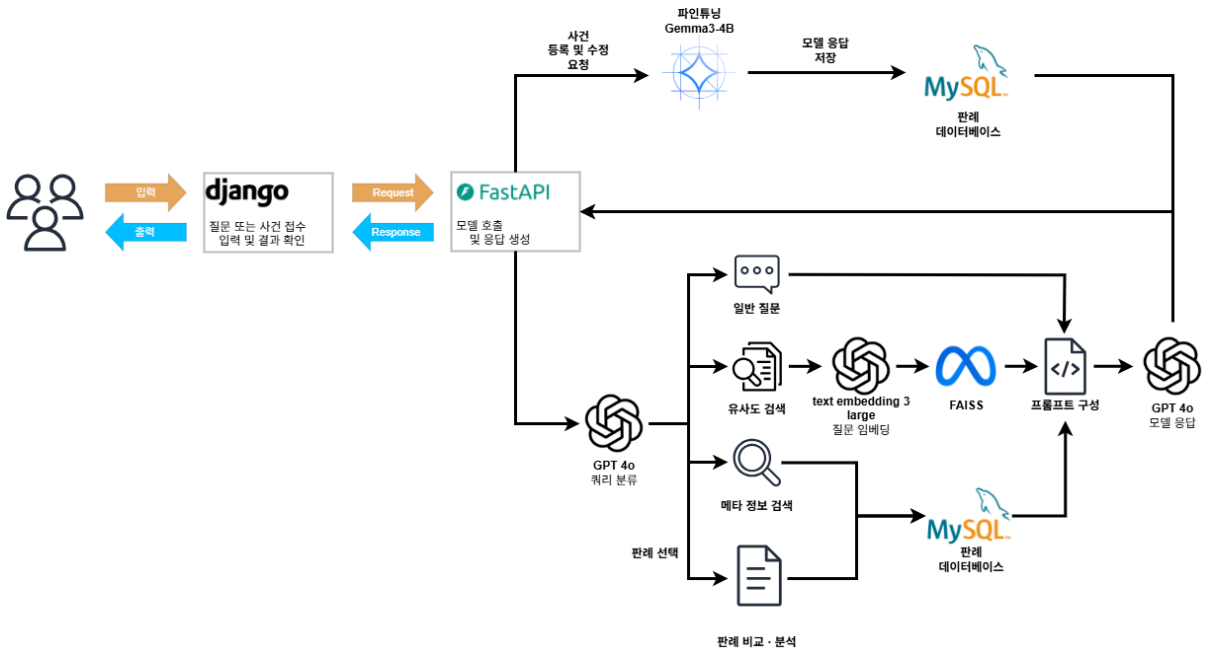
[4] 응답 반환 및 저장

├ 사용자에게 채팅 형식 응답 전송 (프론트로 전달)

▼

[5] 사용자

└ 웹 UI에서 결과 확인 및 후속 질문 가능



3. 코드 구조 **(**모듈화**,** 주석 포함**)**

```
fastapi_server/
├── main.py                    # Entry point
├── vectordb/                  # FAISS 저장 폴더
│   ├── faiss_index.idx        # faiss index 파일
│   └── faiss_index.pkl        # faiss pkl 파일
├── schema/                    # Request, Response 스키마 폴더
│   ├── __init__.py
│   ├── chatbot_schema.py      # 챗봇(판례) 스키마 정의 코드
│   └── strategy_schema.py     # 사건 전략 스키마 정의 코드
├── agent/                     # 에이전트 폴더
│   ├── __init__.py
│   ├── chatbot.py             # 챗봇(판례) 서비스 폴더
│   │   ├── classify_llm.py    # 입력 쿼리 분류 코드
│   │   ├── embedding.py.py    # 쿼리 임베딩 생성 코드
│   │   ├── rds_query.py       # RDB 조작 코드
│   │   └── vector_search.py   # VectorDB 유사도 검색 코드
│   └── strategy_schema.py     # 사건 전략 서비스 코드
├── services/                  # 서비스 폴더
│   ├── __init__.py
│   ├── strategy.py            # 사건 전략 서비스 코드
│   └── case.py                # 판례
├── db/                        # RDS(MySQL) 호출 폴더
│   ├── __init__.py
│   ├── case_db.py             # case(판례) 테이블 호출 코드
│   └── event_db.py            # event(사건) 테이블 호출 코드
├── models/                    # AI 모델 호출 폴더
│   ├── __init__.py
│   ├── response_format.py     # 모델 응답의 포맷을 맞추어주기 위한 코드
│   ├── vllm.py                # Runpod serverless vLLM 모델 호출 코드
│   └── gpt.py                 # GPT 모델 호출 코드
├── .dockerignore              # 도커 예외처리 파일
├── Dockerfile                 # 도커 파일
└── requirements.txt
```

※ 추후 이 포멧으로 모듈화 예정

4. 주요 코드 **(**요약 **+** 평가요소 적용**)**

```python
@app.post("/add-strategy/", response_model=EventResponse)
async def analyze_case(case: EventRequest):
    try:
        logger.info(f"분석 요청 받음: {case.client_role}")
        strategy = generate_strategy(case)
        return EventResponse(result=strategy)
    except Exception as e:
        logger.error(f"분석 중 오류 발생: {str(e)}")
        raise HTTPException(status_code=500, detail=f"분석 중 오류가 발생했습니다: {str(e)}")

@app.post("/update-strategy/", response_model=EventResponse)
async def update_ai_strategy(case: EventRequest, db: Session = Depends(get_db)):
    try:
        logger.info(f"분석 요청 받음: {case.client_role}")
        strategy = generate_strategy(case)
        event = db.query(Event).filter(Event.event_id == case.event_id).first()
        event.ai_strategy = strategy
        db.commit()
        return {"message": f"AI 전략이 성공적으로 생성되었습니다. (Event ID: {case.event_id})"}

    except Exception as e:
        logger.error(f"Request body 파싱 실패: {str(e)}")
        raise HTTPException(status_code=500, detail=f"분석 중 오류가 발생했습니다: {str(e)}")
```

```python
@router.post("/combined", response_model=UnifiedResponse)
async def unified_search(request: UnifiedQueryRequest):
    """통합 검색 엔드포인트 - 모든 기능을 하나로 통합"""
    try:
        query = request.query
        case_id = request.case_id
        case_ids = request.case_ids or []

        # case_id가 제공된 경우: 단일 판례 질문
        if case_id:
            return await handle_single_case_query(query, case_id)

        # case_ids가 제공된 경우: 특정 판례들에 대한 질문
        if case_ids:
            return await handle_specific_cases_query(query, case_ids)

        # case_ids가 없는 경우: 일반 검색 (조건/유사도/일반질문)
        return await handle_general_search(query)

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

4.1 예외 처리 포함

- query, case_id, case_ids 등 사용자 입력값이 다양한 경우의 수를 가질 수 있어, 조건 분기에 따라 호출되는 내부 처리 함수도 달라집니다.

- 각 처리 함수 내부에서 추가적인 LLM 호출, 벡터 DB 검색, RDS 질의 등이 일어나므로, 이 전체를 한 번의 try 블록으로 감싸 상황별 예외를 처리할 수 있도록 설계되어 있습니다.

4.2 .env & 환경변수 설정

- 본 시스템은 개발 환경과 배포 환경 모두에서 설정 값을 관리하고, 민감한 정보를 보호하기 위해 환경변수 기반 구성을 사용하였습니다.

- 특히 Runpod과 같은 배포 환경에서는 .env 파일을 별도로 사용하지 않고, Public Environment Variables 항목에 필요한 값을 직접 등록하여 사용합니다.

- 이 값들은 런타임 시 자동으로 시스템 환경변수로 인식되도록 설계되어 있어, 코드에 민감한 정보를 포함하지 않고도 안정적으로 운영할 수 있습니다.

5. 프롬프트 최적화

```
def generate_markdown_answer(prompt: str) -> str:
    """마크다운 형식으로 답변 생성"""
    markdown_prompt = f"""
{prompt}

**중요**: 반드시 마크다운 형식으로 답변해주세요.

**마크다운 형식 규칙:**
1. **제목**: ### 사용 (예: ### 주요 쟁점)
2. **강조**: **굵게** 사용 (예: **중요한 내용**)
3. **목록**: - 사용 (예: - 첫 번째 항목)
4. **법조항**: `백틱` 사용 (예: `민법 제750조`)
5. **구분선**: --- 사용 (필요시)

**답변 구조:**
- 명확한 제목으로 시작
- 중요 내용은 굵게 표시
- 목록으로 정리
- 구조화된 형태로 작성

마크다운 형식으로 답변해주세요:
"""
```

웹 UI에서 사용자에게 답변을 보여줄 때, 그냥 긴 텍스트로 출력하면 가독성이 떨어지기 때문에, 중요한 내용은 굵게, 핵심 항목은 목록으로, 법조문은 백틱으로 감싸는 형식으로 출력되도록 마크다운 형식을 따로 지정해서 포맷팅하도록 구성하였습니다. 이렇게 하면 사용자 입장에서 정돈된 형태로 판례 답변을 확인할 수 있게 되고, 개별 항목도 한눈에 볼 수 있게 됩니다.

6. 테스트 시나리오

input

{

*"query": "*민법 제*393*조, 제*763*조 내용이 어떻게 돼*?"*

}

output

{

*"answer": "*민법 제*393*조와 제*763*조는 손해배상과 관련된 조항입니다.*\n\n-* **민법 제*393*조**:* 이 조항은 채무불이행으로 인한 손해배상의 범위에 대해 규정하고 있습니다. 채무자가 채무를 이행하지 않거나 불완전하게 이행함으로 인해 채권자에게 발생한 손해를 배상하도록 정하고 있습니다. 손해배상의 범위는 채무불이행으로 인해 통상적으로 발생하는 손해로 제한됩니다.*\n\n-* **민법 제*763*조**:* 이 조항은 불법행위로 인한 손해배상을 규정한 조항입니다. 구체적으로는 제*750*조부터 제*762*조 사이에 있는 불법행위에 관한 규정이 제*393*조의 채무불이행에 관한 규정을 준용하도록 정하고 있습니다. *\n\n*이 두 조항은 손해배상과 관련되어 있으며, 특히 손해의 범위와 배상액에 관한 원칙을 규정하고 있습니다."

}

→ 벡터 검색 + LLM 연동 + 예외 처리 + 최적화된 응답 확인

★ 평가요소표

평가 항목	대응 내용
적용 위치	`/combined` API (모든 검색/응답 기능이 집중된 핵심 엔드포인트)
예외 처리 방식	`try-except` 구문으로 모든 내부 처리 흐름을 감싸고, 실패 시 `HTTPException`으로 일관된 500 응답 반환
안정성 확보	예외 발생 시에도 서버 중단 없이 오류 메시지를 명시적으로 전달하여 서비스 가용성 유지
에러 대응 구조화	단일 질문, 복수 판례 질문, 일반 검색 등 조건별 흐름을 사전에 분기하여 예외 가능성을 최소화함
보안 및 디버깅 대비	`detail=str(e)` 형태로 구체적인 예외 메시지를 로그로 확보 가능 (운영 시에는 조정 가능)